

MORK, MM2, and PathMap: A Lean 4 Formalization

Syntax, Space Semantics, Trie Data Structure, Zipper Execution Model,
OSLF Language Instance, Evidence Bridges, Conformance Testing,
and Cross-System Bridges

Zar Oruži*

March 2026

Abstract

We present a Lean 4 formalization of the MORK execution engine, its MM2 (Minimal MeTTa 2) language, and the PathMap trie data structure that underpins it. The formalization covers MM2 syntax, space semantics with pattern matching (including recursive expression matching), a three-phase priority-scheduled execution protocol, the PathMap algebraic lattice hierarchy with a concrete finite trie instance (`FTrieV`) satisfying `PathMapQuantale`, a bridge to MeTTaIL’s declarative reduction semantics, a bridge to MQ-calculus COMM non-determinism, a Z3 oracle semantics module, an invertible compat-head fragment compiler for PeTTa evaluation-during-matching, and a direct-exec surface packaging scheduler-level exactness. The PathMap library provides a six-class zipper typeclass hierarchy mirroring the Rust `pathmap::zipper` trait stack, with a Zipper Abstract Machine (ZAM) soundness theorem, a complete OSLF language instance with NTT soundness covering all four algebraic operations via 10 conditional rewrite rules, `PLN BinaryEvidence` bridges with Knuth–Skilling valuations, Solomonoff semimeasure weighting, and a `PathMapValuation` typeclass with counting and weighted instances. A conformance test suite of kernel-checked theorems is verified against the MORK CLI (`mork run`) on 9+ `.mm2` test programs. The MORK/MM2 development comprises 12,373 lines across 19 files with 0 sorries and 0 custom axioms. The PathMap library adds 7,165 lines across 27 files with 160+ theorems and 40+ instances, including a Huet zipper with navigation round-trips, catamorphism/anamorphism framework, graft correctness, and a candidate-selection architecture for HE backends. The combined development totals 19,538 lines across 46 files. All proofs are checked by Lean 4.28 with Mathlib v4.28.

Contents

1	Introduction	1
2	MM2 Syntax	2
2.1	Fold-Level Aggregation	4
3	Space Semantics and Matching	4
3.1	Space and Substitution	4
3.2	Pattern Matching	5
3.3	Sink Application and Rule Firing	5
3.4	Structural Lemmas	5

*“Oruži” denotes AI-assisted collaboration using Claude Code (Anthropic), ChatGPT, and Codex (OpenAI).

4	Match Specification	6
5	Three-Phase Execution	6
6	Work-Queue Scheduler	6
6.1	Exec-Fact Extraction	7
6.2	Ordering	7
6.3	Ground Self-Respawn	7
6.4	Scheduler-Level Correspondence	8
6.5	Source-Side Input Semantics	9
7	Sink Persistence	9
8	PathMap Algebraic Hierarchy	9
8.1	AlgebraicResult and Typeclass Hierarchy	9
8.2	Finset Instance	10
8.3	Coinductive Trie (CTrie)	10
8.4	Finite Trie (FTrie)	10
8.5	PathMapQuantale Instance for FTrie	11
9	PathMap Zipper Hierarchy and ZAM Soundness	11
9.1	ZAM Soundness	12
9.2	Concrete Instances	12
10	PathMap OSLF Language Instance	12
11	PathMap Evidence Bridges	13
11.1	PLN BinaryEvidence (PLNBridge.lean, 275 lines)	13
11.2	Solomonoff Weighting (SolomonoffBridge.lean, 239 lines)	13
11.3	Valuations and World Model (Measure.lean, 143 lines; WorldModelBridge.lean, 151 lines)	14
12	MORK–PathMap Bridge	14
12.1	Candidate Selection, Support, and Counts	15
13	MeTTaIL Bridge	15
13.1	Zero-Premise Bridge	15
13.2	Multi-Premise Bridge via PremiseChain	16
13.3	Freshness Guards and the Extended Bridge	16
13.4	Concrete PLN Rules as GSLT Routing Witnesses	17
13.5	Authored Context and Collection-Shaped Lens Bridge	17
13.6	Compound Pattern Round-Trips	19
13.7	Extended Bridge: Beyond morkTranslatable	19
14	MQ-Calculus Bridge	20
14.1	N-ary Generalization	20
15	Z3 Oracle Semantics	20

16 Invertible Compat-Head Fragment	21
16.1 Problem	21
16.2 Key Insight	21
16.3 Formalization	21
17 Direct-Exec Surface	22
17.1 Scheduler Seam	22
17.2 PeTTa Fragment Bridge	23
18 Conformance Testing	23
18.1 Computable Reference Evaluator	23
18.2 Test Fixtures	23
18.3 Expression Matching	23
19 Coverage and Open Gaps	24
19.1 Notable Restrictions	24
20 Execution Boundary	25
21 Runtime-Kernel Packaging	26
21.1 Effect-Class Bridge (<code>RuntimeKernel.lean</code>)	26
21.2 Resource Layer (<code>RuntimeResource.lean</code>)	26
22 Storage vs Computation Cost Analysis	29
22.1 Crossover Theorem	29
22.2 Reverse Query Advantage	29
22.3 Concrete Arithmetic Analysis	29
22.4 Trie vs Flat Table	30
22.5 Formalized Files	30

1 Introduction

MORK (“MeTTa Optimal Reduction Kernel”) is a parallel metagraph rewriting engine that executes programs written in MM2, a forward-chaining term-rewriting language over spaces of atoms [1]. MORK uses PathMap as its underlying data structure for efficient graph and path representation, and serves as the execution substrate for MeTTa implementations.

This note documents a Lean 4 formalization of the MORK/MM2/PathMap stack, covering:

1. MM2 syntax: exec rules with prioritized pattern-template pairs, including **head** (idempotent add) sinks and fold-level aggregators (§2);
2. Space semantics: finite sets of ground atoms with substitution-based pattern matching and sink application (§3);
3. A relational match specification with soundness and completeness (§4);
4. A three-phase execution protocol (unfold/base/fold) with priority-band ordering and fold-level aggregation (§5);
5. A faithful work-queue scheduler with read-copy semantics, exec-fact extraction, and ground self-respawn (§6);
6. Sink persistence: atoms persist through sink pipelines unless explicitly removed (§7);

7. The PathMap algebraic hierarchy: lattice, distributive lattice, and quantale, with a concrete finite trie instance (§8);
8. Algebraic correspondence between MORK space transitions and PathMap join/subtract (§12);
9. A MeTTaIL bridge: zero-premise, multi-premise, freshness-guarded, and authored contextual reductions imply MORK rule firings. One-hole contexts use a Huet zipper/lens factorization (`LensRel`), while collection shape alone grants no reduction authority, with concrete PLN inference rules as end-to-end routing witnesses (§13);
10. An MQ-calculus bridge: binary and N-ary fold non-determinism corresponds to `CommReduction` (§14);
11. A PathMap zipper typeclass hierarchy with six classes (`ZipperMoving...ZipperComplexity`) mirroring the Rust `pathmap::zipper` trait stack, with ZAM (Zipper Abstract Machine) soundness (§9);
12. A complete PathMap OSLF language instance with 10 conditional rewrite rules and NTT completeness biconditionals for all four algebraic operations (§10);
13. PLN `BinaryEvidence` bridges from PathMap stores with Knuth–Skilling valuations, Solomonoff semimeasure weighting, and a `PathMapValuation` typeclass (§11);
14. Z3 oracle semantics: `OracleEnv`, payload-to-space conversion, and oracle-level pattern matching (§15);
15. An invertible compat-head fragment formalizing the compilation of PeTTa evaluation-during-matching to standard MORK three-phase execution via staged inverse compilation (§16);
16. A direct-exec surface packaging scheduler-level exactness with PeTTa eval fragment composition (§17);
17. A conformance test suite verified against the MORK CLI (§18);
18. An execution boundary packaging (`morkTranslatable`) that delineates the proven fragment (§20).

Tables 1 and 2 summarize the formal development.

All MORK/MM2 files reside under `Mettapedia/Languages/ProcessCalculi/MORK/`. The PathMap library is at `Mettapedia/OSLF/PathMap/`.

2 MM2 Syntax

MM2 programs consist of *exec rules* that match atoms in a space and produce add/remove mutations. The grammar, as formalized in Lean following the MORK wiki spec, is:

```

ExecRule ::= (exec (priority n name) Pattern Template)
Pattern  ::= (, Atom1 ... Atomn)
Template ::= (0 Sink1 ... Sinkn)
Sink     ::= (+ Atom) | (- Atom) | (head Atom)

```

A *pattern* is a conjunction: all atoms must be present in the space for the rule to fire. A *template* is a list of sinks applied simultaneously: `(+ a)` inserts atom `a`, `(- a)` removes it, and `(head a)` performs an idempotent add (first-wins: no-op if already present).

The Lean formalization uses the shared `Atom` type from `MeTTa.Core.Atom`, so MORK S-expressions are a subset of MeTTa atoms:

Listing 1: MM2 syntax types (`Syntax.lean`).

```

structure Pattern where
  atoms : List Atom
  deriving Repr, DecidableEq

```

Table 1: MORK/MM2 formalization files.

File	Key Content	Lines
Syntax.lean	Sink, FoldAggregator, ExecRule, SourceFactor, InputSpec	318
Space.lean	Space, matchAtom, applySink, source-aware matching	432
MatchSpec.lean	MatchAtomRel, soundness/completeness	133
ThreePhaseExec.lean	Phase protocol, aggregator theorems, canaries	381
ThreePhaseRefinement.lean	Sink persistence, phase $\leftrightarrow$ scheduler, exactness	519
WorkQueueOrder.lean	atomKey, shortlex ordering, location-based exec ordering	561
WorkQueueExec.lean	Read-copy scheduler, source-aware firing, correspondence	1,192
MORKCommBridge.lean	MQ correspondence, N-ary generalization	300
PathMapBridge.lean	Base/fold = pjoin/psubtract	174
MeTTaILBridge.lean	Multi-premise bridge, PremiseChain, ext bridge	1,430
PLNRules.lean	Concrete PLN rules, guarded bridge instantiation	378
Conformance.lean	Computable evaluator, source conformance, correspondence	1,694
AtomZipper.lean	Huet zipper, LensRel, lens laws, collection specialization	283
AuthoredContextBridge.lean	Authored one-hole contexts, exact MORK rule compilation	321
CollectionBridge.lean	Collection-shaped lens specialization and negative boundary	80
ExtendedBridge.lean	Extended bridge for .subst + rest-variable resolution	83
ExecutionBoundary.lean	morkTranslatable + scheduler + source + ext re-exports	510
Z3Oracle.lean	Z3 oracle semantics, payload-to-space, conformance	347
DirectExecSurface.lean	Scheduler exactness package, PeTTa fragment bridge	234
InvertibleHead.lean	PeTTa compat-head compilation, staged inverse soundness	2,766
Total		12,373

```

inductive Sink where
| add      : Atom -> Sink   -- (+ a)
| remove   : Atom -> Sink   -- (- a)
| head     : Atom -> Sink   -- (head a): idempotent add
deriving Repr, DecidableEq

structure Template where
sinks : List Sink
deriving Repr, DecidableEq

structure ExecRule where
priority : Nat
name      : String
pat       : Pattern
tmpl      : Template
deriving Repr, DecidableEq

```

2.1 Fold-Level Aggregation

Sub-results assembled during the fold phase are governed by a `FoldAggregator`:

Listing 2: Fold aggregator (Syntax.lean).

```

inductive FoldAggregator where
| selectAll    : FoldAggregator -- non-deterministic (default)
| selectFirst  : FoldAggregator -- head only
| count        : FoldAggregator -- (.grounded (.int N))
| sum          : FoldAggregator -- sum of integer sub-results
deriving Repr, DecidableEq

def applyAggregator (agg : FoldAggregator)
  (subResults : List Atom) : Option Atom :=
  match agg with
  | .selectAll    => subResults.head?
  | .selectFirst  => subResults.head?
  | .count        =>
    some (.grounded (.int subResults.length))
  | .sum          =>
    let vals := subResults.filterMap extractInt
    some (.grounded (.int (vals.foldl (. + .) 0)))

```

The `selectAll` and `selectFirst` cases both return `head?` at this level; the non-determinism of `selectAll` is captured by `NaryFoldPicksSubResult` in `MORKCommBridge.lean`, not by `applyAggregator`.

Theorem 2.1 (`applyAggregator_count`). *`applyAggregator .count rs = some (.grounded (.int rs.length))`.*

Theorem 2.2 (`applyAggregator_selectFirst`). *For `rs = a :: tl`: `applyAggregator .selectFirst rs = some a`.*

Theorem 2.3 (`applyAggregator_sum_nil`). *`applyAggregator .sum [] = some (.grounded (.int 0))`.*

3 Space Semantics and Matching

3.1 Space and Substitution

A MORK *space* is a finite set of ground atoms. The Rust implementation uses `PathMap<()>`; the formalization abstracts this to `Finset Atom` for tractable reasoning:

Listing 3: Space and substitution (`Space.lean`).

```

abbrev Space := Finset Atom
abbrev Subst := List (String * Atom)

```

Substitutions are association lists with first-match-wins lookup via `Subst.lookup`.

3.2 Pattern Matching

The spec-level function `matchAtom` implements first-order (non-unification) pattern matching on three atom kinds:

- **Variable** (`.var v`): if `v` is already bound in the substitution, the concrete atom must equal the bound value; otherwise, `v` is freshly bound.
- **Symbol** (`.symbol s`): must match an identical symbol.
- **Grounded** (`.grounded g`): must match an identical grounded value.
- **Expression** (`.expression es`): returns `none` in the spec-level `matchAtom` (see §18 for the computable evaluator that handles expressions).

The conjunctive match `matchPattern` threads the substitution through a list of pattern atoms, matching each against the space while tracking consumed atoms to prevent double-matching.

3.3 Sink Application and Rule Firing

A single sink mutates the space:

Listing 4: Sink application (`Space.lean`).

```
def applySink (s : Space) (sig : Subst) (sink : Sink)
  : Space :=
  match sink with
  | .add a    =>
    let a' := applySubst sig a
    if isGroundAtom a' then s ∪ {a'} else s
  | .remove a => s.erase (applySubst sig a)
  | .head a   =>
    let a' := applySubst sig a
    if isGroundAtom a' then s ∪ {a'} else s
```

Non-ground add results are silently dropped (a guard for soundness). In the `Finset` model, `head` is definitionally equal to `add` (union is idempotent). The distinction is operative in the computable reference evaluator (`Conformance.lean`), which uses `List Atom` where idempotence must be checked explicitly via `contains`. A template’s sinks are applied left-to-right via `applySinks`, and `fireRule` composes matching with template application.

3.4 Structural Lemmas

Four theorems characterize sink effects:

Theorem 3.1 (`applySink_add_card`). *Adding a ground atom not already in the space increases cardinality by 1.*

Theorem 3.2 (`applySink_remove_card`). *Removing an atom present in the space decreases cardinality by 1.*

Theorem 3.3 (`applySinks_empty`). *An empty template is the identity on the space.*

Theorem 3.4 (`applySink_add_mem`). *Adding an atom preserves membership of all existing atoms.*

4 Match Specification

The file `MatchSpec.lean` introduces a relational specification for pattern matching, separating the “what” from the “how”:

Listing 5: Match relation (`MatchSpec.lean`).

```
inductive MatchAtomRel : Subst -> Atom -> Atom
  -> Subst -> Prop where
  | var_fresh (hlookup : sig.lookup v = none) :
    MatchAtomRel sig (.var v) a ((v, a) :: sig)
  | var_bound (hlookup : sig.lookup v = some a) :
    MatchAtomRel sig (.var v) a sig
  | symbol : MatchAtomRel sig (.symbol s) (.symbol s) sig
  | grounded : MatchAtomRel sig (.grounded g) (.grounded g) sig
```

The key result is an exact characterization:

Theorem 4.1 (`matchAtom_iff`). *For all substitutions σ , pattern atoms p , concrete atoms c , and result substitutions σ' : $\text{matchAtom } \sigma \ p \ c = \text{some } \sigma' \iff \text{MatchAtomRel } \sigma \ p \ c \ \sigma'$*

This is proved via separate soundness and completeness theorems. Expression patterns are excluded: `matchAtom_expression_pattern_none` proves that `matchAtom` on `.expression` always returns `none`.

5 Three-Phase Execution

MORK’s execution protocol schedules rules by priority into three non-overlapping bands:

Definition 5.1 (Phase Bands).

Phase	Priority Range	Role
Unfold	0–31	Spawn N sub-queries + wait token
Base	32–63	Resolve leaf queries directly
Fold	64–95	Wait for sub-results, assemble final result

Each phase has a dedicated step type (`UnfoldStep`, `BaseStep`, `FoldStep`) with a proof field witnessing that the step’s priority falls within the correct band. The `FoldStep` structure carries an `aggregator : FoldAggregator` field (default `.selectAll`) governing how sub-results are assembled. Space transitions `applyUnfold`, `applyBase`, and `applyFold` implement each phase.

Theorem 5.2 (`phase_ranges_disjoint`). *No priority belongs to two distinct phases.*

Theorem 5.3 (`phase_priority_monotone`). *For any priorities n_u , n_b , n_f in the unfold, base, and fold phases respectively: $n_u < n_b < n_f$.*

Theorem 5.4 (`foldStep_default_aggregator`). *For a fold step with default aggregator (`.selectAll`): $\text{applyAggregator } \text{fold.aggregator } \text{fold.subResults} = \text{fold.subResults.head?}$.*

6 Work-Queue Scheduler

The work-queue scheduler (`WorkQueueExec.lean`, 1,192 lines) formalizes MORK’s actual runtime model from `mork_backend.rs`. This replaces the abstract three-phase view with the concrete execution loop:

1. **Pop** the next (`exec ...`) fact from the space (sorted by `atomKey`: serialized key with shortlex ordering).
2. **Remove** it from live space.
3. **Read copy**: `live_after_remove` $\cup$ `{exec_fact}`—re-insert so the rule can match itself.
4. **Match** the pattern against the read copy.
5. **Apply** template sinks to live space (not to read copy).
6. **Repeat** until no `exec` facts remain or step limit reached.

The read-copy semantics are critical: step 3 ensures that a rule can match its own `exec` fact (required for self-referential patterns).

Theorem 6.1 (`readCopy_mem_exec`). *The `exec` fact is always a member of its own read copy.*

Theorem 6.2 (`readCopy_eq_of_mem`). *Remove + re-insert = identity on membership: if $a \in s$ then $a \in (s \setminus \{a\}) \cup \{a\}$.*

6.1 Exec-Fact Extraction

The function `extractExecFact` parses an `(exec loc pat tpl)` atom into a structured `ExecFact` with priority, name, pattern, and template fields. Sink parsing handles `(+ ...)`, `(- ...)`, and `(head ...)` forms.

6.2 Ordering

`WorkQueueOrder.lean` (561 lines) defines the `atomKey` serialization and `lexLt` comparison used to determine exec-fact firing order:

Theorem 6.3 (`order_p0_lt_p1`). *`atomKey` orders priority 0 before priority 1.*

Theorem 6.4 (`order_1_lt_10`). *Shortlex ordering: single-digit priorities fire before double-digit.*

Byte-order refinement. `PathMapByteOrderRefinement.lean` closes the gap noted in `WorkQueueOrder.lean`: “This is NOT byte-identical to PathMap serialization.” On the scheduler exec-location fragment (`schedulerLocFragment`: two-symbol expressions), the theorem `scheduler_byte_faithful` proves that the abstract ordering `lexLt ∘ atomKey` agrees with the concrete PathMap byte-order traversal used by the Rust `metta_calculus`. The companion theorem `scheduler_pop_order_correct` says: if `locLt loc1 loc2 = true`, then the abstract key ordering confirms `loc1` is popped first. The unsupported remainder (nested expressions, non-ASCII, length ≥ 64) is stated explicitly.

6.3 Ground Self-Respawn

A canary theorem exercises the scheduler with a ground rule that re-adds its own exec fact via a `(+ ...)` sink:

Theorem 6.5 (`canary8_ground_self_respawn`). *A ground self-respawn rule fires exactly once—the second step finds no new match since the exec fact is already present in the space.*

Theorem 6.6 (`canary8_second_step_no_match`). *Running the scheduler for 2 steps on the self-respawn rule yields the fired result plus the original exec fact, with step count 2.*

6.4 Scheduler-Level Correspondence

The computable work-queue (`cWorkQueueStep`, `cWorkQueueRunN`) operates on `List Atom`, while the spec-level definitions (`workQueueStep`, `workQueueRunN`) operate on `Finset Atom`. The scheduler correspondence chain establishes that these agree at the `toFinset` level, building from per-piece soundness through full bounded-run equivalence.

Per-sink correspondence. Each computable sink step (`capplySinkStep`) agrees with the spec-level `applySink`, provided a `NodupSafe` invariant holds (no duplicates at each remove step during the foldl over sinks). This is the base layer.

Multi-match foldl correspondence. The general case lifts per-sink correspondence through the outer foldl over match results. The predicate `FoldNodupSafe` ensures `NodupSafe` at every step of this outer fold:

Definition 6.7 (`FoldNodupSafe`). Given an accumulator `acc`, a list of match results `ms`, and a template `tmpl`: `FoldNodupSafe(acc, ms, tmpl)` holds iff for every $i < |ms|$, the accumulator after folding the first i results satisfies `NodupSafe` for the i -th substitution and the template’s sinks.

Theorem 6.8 (`foldl_capplySinks_toFinset`). *If the computable and spec match results have the same substitutions in the same order and `FoldNodupSafe` holds, then the foldls produce corresponding results at the `Finset` level.*

The alignment hypothesis (same substitutions in same order) is explicit because `Finset.toList` has unspecified order. For patterns that match at most once (the common MORK case), this is trivially satisfied.

General firing correspondence.

Theorem 6.9 (`cFireExecFact_toFinset`). *For any exec fact ef with $ef.atom \in s$, if match results align and `FoldNodupSafe` holds, then $cFireExecFact(s, ef).toFinset = fireExecFact(s.toFinset, ef)$.*

Step and bounded-run correspondence.

Theorem 6.10 (`cWorkQueueStep_toFinset`). *If the `WorkQueueInvariant` holds (bundling `Nodup`, `KeyInjective`, and per-firing alignment), then the computable and spec work-queue steps agree: $(cWorkQueueStep\ s).map\ toFinset = workQueueStep\ (s.toFinset)$.*

Theorem 6.11 (`cWorkQueueRunN_toFinset`). *If the `WorkQueueInvariant` holds at every computably reachable state (formalized via the `CReachable` inductive), then n -step bounded execution agrees:*

$$(cWorkQueueRunN\ n\ s).1.toFinset = (workQueueRunN\ n\ (s.toFinset)).1 \quad \text{and} \quad (cWorkQueueRunN\ n\ s).2 = (workQueueRunN\ n\ (s.toFinset)).2$$

The invariant-maintenance burden is explicit: for finite-state MORK programs (the common case), it can be verified by exhaustive enumeration.

6.5 Source-Side Input Semantics

MORK exec rules support two input modes: *compat mode* $(, pat_1 \dots pat_n)$, where all pattern atoms are matched against the workspace, and *explicit source mode* $(I\ src_1 \dots src_n)$, where each source factor specifies how its pattern is matched.

The formalization covers three source factor types (`SourceFactor` in `Syntax.lean`):

- **btm**: Match pattern against the workspace (equivalent to compat-mode per-atom matching).
- **eqConstraint** (`==`): Substitute the first sub-expression using current bindings, check if the result exists in the space, and if so bind the witness to the found atom.
- **neqConstraint** (`!=`): Substitute the first sub-expression, *remove* it from the candidate set, then match the witness against remaining atoms. This implements the Rust `CmpSource` with `cmp=1`.

Theorem 6.12 (`fireSourceRule_compat`). *Source-aware firing on a compat-mode rule agrees with the standard `fireRule`: $fireSourceRule\ s\ (r.toSourceRule) = fireRule\ s\ r$.*

The computable evaluator (`cmatchInputSpec`, `cfireSourceRule`, `cFireSourceExecFact`) mirrors the spec-level definitions, and 11 kernel-checked conformance tests (tests S1–S8 in `Conformance.lean` plus canaries 9–11 in `WorkQueueExec.lean`) verify the implementation against `mork run` ground truth, covering BTM matching, equality constraint success/failure, shared variable propagation, inequality exclusion, and multi-match non-determinism.

7 Sink Persistence

`ThreePhaseRefinement.lean` (519 lines) proves that atoms persist through sink pipelines unless explicitly removed, and provides `phase↔scheduler` exactness theorems (`base_step_exactness`, `unfold_step_exactness`, `fold_step_exactness`):

Theorem 7.1 (`applySink_mem_of_mem`). *If $b \in s$ and the sink is not a remove of b (after substitution), then b remains after applying the sink. Handles all three sink constructors: `add`, `remove`, and `head`.*

Theorem 7.2 (`applySinks_mem_of_mem`). *If $b \in s$ and no sink in the template removes b , then $b \in \text{applySinks } s \ \sigma \ \text{tpl}$.*

The proof proceeds inductively via a helper `foldl_applySink_mem` that threads the non-removal invariant through the `List.foldl` over sinks.

Theorem 7.3 (`applySink_add_ground_mem`). *Adding a ground atom via `Sink.add` guarantees the substituted atom is a member of the resulting space.*

8 PathMap Algebraic Hierarchy

`PathMap` is a trie-indexed data structure with algebraic operations that form a quantale. The formalization provides both the abstract algebraic hierarchy and a concrete trie implementation.

8.1 AlgebraicResult and Typeclass Hierarchy

Operations on `PathMap` values return an `AlgebraicResult V`, an inductive type that encodes structural sharing:

Listing 6: `AlgebraicResult` (`Core.lean`).

```
inductive AlgebraicResult (V : Type u) where
| none : AlgebraicResult V
| identity (isSelf : Bool) (isOther : Bool)
  : AlgebraicResult V
| element : V -> AlgebraicResult V
```

The `identity` variant avoids allocation when the result equals one of the inputs. Three typeclasses form the hierarchy:

- **`PathMapLattice`** α : `pjoin` (union) and `pmeet` (intersection).
- **`PathMapDistributiveLattice`** α : adds `psubtract` (set difference).
- **`PathMapQuantale`** α : adds `prestrict` (prefix filter).

Algebraic law classes (`JoinComm`, `JoinIdem`, `MeetComm`, `MeetIdem`, `Absorption`) are defined at the value level.

8.2 Finset Instance

`Core.lean` provides a `PathMapQuantale (Finset  $\alpha$ )` instance with all algebraic laws proved:

- `pjoin` = union with identity detection ($a = b$, $a \subseteq b$, $b \subseteq a$).
- `pmeet` = intersection; returns `.none` if empty.
- `psubtract` = set difference; returns `.none` if empty.
- `prestrict` = subset check then intersection.

This is the instance used by `PathMapBridge.lean` for `Space = Finset Atom`.

8.3 Coinductive Trie (CTrie)

The file `CoinductiveTrie.lean` defines the semantic reference model:

Listing 7: Coinductive trie (`CoinductiveTrie.lean`).

```
abbrev CTrie V := List UInt8 -> Option V
```

A trie IS its lookup function. Pointwise operations (`union`, `inter`, `diff`, `restrict`) are defined by lifting `Option` combinators, and 30 theorems prove bisimilarity laws (idempotence, commutativity, identity elements, associativity).

8.4 Finite Trie (FTrie)

The concrete, computable trie:

Listing 8: Finite trie (`FiniteTrie.lean`).

```
inductive FTrie (V : Type u) where
| empty : FTrie V
| node (val : Option V)
    (children : List (UInt8 * FTrie V))
    : FTrie V
```

Operations (`lookup`, `join`, `meet`, `subtract`, `restrict`) use mutual recursion with child-list helpers. A `Sorted` predicate (`TrieRefinement.lean`) ensures child keys are ordered, and refinement theorems prove that `FTrie` operations commute with `CTrie` operations:

Theorem 8.1 (`join_lookup`). $(t_1.\text{join } t_2).\text{lookup } p = t_1.\text{lookup } p < | > t_2.\text{lookup } p$ (for *Sorted tries*).

8.5 PathMapQuantale Instance for FTrie

`TriePathMapInstance.lean` provides:

Listing 9: `PathMapQuantale` instance for `FTrie`.

```
instance : PathMapQuantale (FTrie V) where
  pjoin a b :=
    if beqFTrie a b then .identity true true
    else .element (a.join b)
  pmeet a b := ... -- similar with emptiness check
  psubtract a b := ...
  prestrict a b := ...
```

`JoinIdem` and `MeetIdem` are proved (via `beqFTrie_refl`). Full commutativity holds for $V = \text{Unit}$ (the MORK use case where `PathMap<()>` stores set membership).

9 PathMap Zipper Hierarchy and ZAM Soundness

The Rust `pathmap` crate exposes a *zipper* [3] cursor model: a cursor navigates a byte-indexed trie, reading or writing values at the current focus, and advancing via depth-first iteration [4]. The Lean formalization mirrors this trait hierarchy with a six-class typeclass stack (`Zipper.lean`, 482 lines):

Listing 10: Zipper typeclass hierarchy (excerpt).

```

class ZipperMoving (Z : Type*) where
  descendFirstByte : Z -> Option Z
  ascend           : Z -> Option Z
  currentPath      : Z -> TriePath
  atLeaf           : Z -> Bool
  atRoot           : Z -> Bool

class ZipperBounded (Z : Type*) [ZipperMoving Z] : Prop where
  ascend_none_at_root : ∀ z, atRoot z = true -> ascend z = none

class ZipperValues (Z : Type*) (V : Type*)
  extends ZipperMoving Z where
  valueAt : Z -> Option V

class ZipperIteration (Z : Type*) [ZipperMoving Z] where
  toNextVal      : Z -> Z × Bool
  descendLastPath : Z -> Z × Bool

class ZipperComplexity (Z : Type*) [ZipperMoving Z]
  [ZipperIteration Z] : Prop where
  descendFirstKPath_depth_le : ... -- path grows by at most k
  toNextVal_depth_bounded    : ... -- depth grows by at most 1

```

The key invariant, encoded by `ZipperBounded`, is that `ascend` never moves above the zipper’s root. The dual property `ZipperLiveness` asserts that ascending from a non-root position always succeeds. `ZipperComplexity` formalizes $O(k)/O(\text{depth})$ structural depth bounds—necessary preconditions for the time-complexity claims in the Rust implementation.

9.1 ZAM Soundness

`ZipperExecution.lean` (216 lines) connects the zipper cursor model to the OSLF engine’s `RelationalSpace` query interface. The central inductive predicate `ZipperReachableValue` defines which values a depth-first iteration can reach:

Listing 11: Reachability and soundness contract.

```

inductive ZipperReachableValue [ZipperMoving Z]
  [ZipperValues Z V] [ZipperIteration Z] :
  Z -> V -> Prop where
| here (hv : valueAt z = some v)
| step (hnext : (toNextVal z).2 = true)
      (hreach : ZipperReachableValue (toNextVal z).1 v)

class ZipperIterationSound (Z V : Type*) ... : Prop where
  reachable_in_store : ∀ root v, atRoot root -> Reachable root v
                      -> v ∈ allValues root
  store_in_reachable : ∀ root v, atRoot root -> v ∈ allValues root
                      -> Reachable root v

```

The main theorems lift this contract to OSLF:

Theorem 9.1 (`zam_oslf_sound`). *If a `ZipperSpace` agrees with a flat `RelationEnv`, then the OSLF reduction relation is identical for both: $\text{langReducesUsing } zs \text{ lang } p \ q \iff \text{langReducesUsing flat lang } p \ q$.*

Theorem 9.2 (`zam_diamond_sound / zam_box_sound`). *The modal operators $\Diamond$ and $\Box$ are preserved by the zipper-to-flat correspondence.*

9.2 Concrete Instances

Two concrete implementations satisfy the full contract:

- **FlatZipper** α (`FlatZipperInstance.lean`, 117 lines): a degenerate list-cursor serving as reference implementation.
- **SimpleTrieZipper** V (`TrieZipper.lean`, 148 lines): wraps `FTrie V` with an entries list, proving `ZipperIterationSound`.

`ZamContracts.lean` (193 lines) provides compositionality theorems for transferring ZAM soundness across cursor transformations.

10 PathMap OSLF Language Instance

`OSLFInstance.lean` (853 lines) instantiates the OSLF pipeline for the PathMap algebraic language, yielding complete NTT (Native Type Theory) soundness theorems [5]. The language definition `pathMapLang` has:

- **Types**: one sort "V" for PathMap node values.
- **Constructors**: four algebraic operations (`PJoin`, `PMeet`, `PSubtract`, `PRestrict`) plus four result constructors (`PUnion`, `PIntersect`, `PDiff`, `PFilter`).
- **Rewrite rules**: 10 conditional rules gated by `relationQuery` premises encoding algebraic conditions:

Premise	Meaning	Used by
"sub" [x,y]	$x \subseteq y$	PJoin, PMeet, PRestrict
"nsub" [x,y]	$\neg(x \subseteq y)$	PJoin
"ndisj" [x,y]	$x \cap y \neq \emptyset$	PMeet
"disj" [x,y]	$x \cap y = \emptyset$	PSubtract

For each operation, the `AlgebraicResult` cases are exhaustively covered:

- **Identity** (`self/other`): a rewrite rule fires, reducing to the operand.
- **Element**: a rewrite rule fires, reducing to the result constructor (e.g., `PUnion`).
- **None**: *no rule fires*—the diamond is vacuously false.

Theorem 10.1 (`pathMap_pjoin_complete` et al.). *For each of the four operations, the NTT completeness theorem is an exact biconditional: $\Diamond_{\text{relEnv}} PJoin(a, b) \phi \iff$ the correct `AlgebraicResult`-determined rule fires. All four theorems (`pjoin`, `pmeet`, `psubtract`, `prestrict`) are proved with 0 sorries.*

The `RelationBridge.lean` module (200 lines) provides the abstract `RelationalSpace` type-class that decouples the OSLF engine from concrete backends, with a canonical embedding `toRelationEnv`.

11 PathMap Evidence Bridges

Three modules connect PathMap stores to probabilistic reasoning frameworks, grounding the evidence semantics for PLN [7] and Solomonoff induction [8].

11.1 PLN BinaryEvidence (PLNBridge.lean, 275 lines)

For a PathMap store $W : \text{Finset } \alpha$ and query $q : \text{Finset } \alpha$, define:

$$\text{finsetPathEvidence } W \ q = (|W \cap q|, |W \setminus q|)$$

where $n^+ = |W \cap q|$ is positive evidence and $n^- = |W \setminus q|$ is negative evidence.

Theorem 11.1 (evidence_total (K&S sum rule)). $n^+ + n^- = |W|$.

Theorem 11.2 (pjoin_evidence_additive). *For disjoint W_1, W_2 : $ev(W_1 \cup W_2, q) = ev(W_1, q) \oplus ev(W_2, q)$.*

Additional theorems cover boundary conditions (evidence_empty_store, evidence_self_query), monotonicity (evidence_monotone), and Bayesian strength interpretation.

Remark 11.3 (Honest scope). Finset union is idempotent ($W \cup W = W$), so full additivity fails. A true BinaryWorldModel instance requires replacing Finset with Multiset (allowing duplicates); this extension is in WorldModelBridge.lean.

11.2 Solomonoff Weighting (SolomonoffBridge.lean, 239 lines)

Generalizes counting evidence to arbitrary weight functions $w : \alpha \rightarrow \mathbb{R}_{\geq 0}^\infty$:

$$\text{weightedPathEvidence } w \ W \ q = \left(\sum_{p \in W \cap q} w(p), \sum_{p \in W \setminus q} w(p) \right)$$

Theorem 11.4 (solomonoffPathEvidence_additive). *Disjoint additivity holds for the Solomonoff semimeasure instantiation.*

Theorem 11.5 (finsetPathEvidence_eq_uniform). *Uniform weight $w \equiv 1$ recovers counting evidence.*

11.3 Valuations and World Model (Measure.lean, 143 lines; WorldModelBridge.lean, 151 lines)

PathMapValuation α is a typeclass with Knuth–Skilling axioms (zero store $\rightarrow$ zero weight, disjoint additivity):

Listing 12: PathMapValuation typeclass (excerpt).

```
class PathMapValuation (α : Type*) [DecidableEq α] where
  weight : Finset α → ℝ≥0∞
  weight_empty : weight ∅ = 0
  weight_disjoint_add : Disjoint W1 W2 →
    weight (W1 ∪ W2) = weight W1 + weight W2
```

Two instances are proved: countingPathMapValuation (weight $W = |W|$) and mkWeightedValuation w (weight $W = \sum_{p \in W} w(p)$). WorldModelBridge.lean lifts Finset evidence to Multiset-based BinaryWorldModel instances where full (non-disjoint) additivity holds.

12 MORK–PathMap Bridge

`Space = Finset Atom` inherits the `PathMapQuantale` instance. The bridge file proves that MORK’s space transitions decompose into algebraic operations:

Theorem 12.1 (`pjoin_singleton_resolves`). $PJ\ s\ \{a\}$ resolves to $s \cup \{a\}$.

Theorem 12.2 (`psubtract_singleton_erase`). $PS\ s\ \{a\}$ resolves (with $\emptyset$ as default) to $s \setminus \{a\}$.

Theorem 12.3 (`applyBase_eq_lattice_ops`). $applyBase = PS$ (remove query) then PJ (add result).

Theorem 12.4 (`applyFold_eq_lattice_ops`). $applyFold = chained\ PS$ (remove wait + sub-results) then PJ (add assembled result).

These theorems confirm that MORK’s operational transitions are expressible as algebraic operations on the PathMap lattice, connecting the execution engine to the data-structural semantics.

Seeded rematch correctness. When the PathMap bridge provides candidate atoms for an imported space, CeTTa rematches each candidate natively with `space_subst_match_with_seed`. The theorem `seedRematch_extends_seed` (`SeedRematchContract.lean`) proves that the resulting bindings extend the incoming seed from prior conjunction steps—prior bindings are never dropped. The companion theorem `seedRematchSafe_no_loop` guarantees that binding loops are detected and rejected before entering the conjunction accumulator. Together with `importedFallbackParity` from `ImportedRowContract.lean`, these establish that CeTTa’s imported-backend path is binding-correct under its current always-rematch policy.

12.1 Candidate Selection, Support, and Counts

Recent companion work on faithful backends for HE MeTTa sharpens what this bridge should and should not claim. A trie such as PathMap is responsible for *support-level* duties:

1. maintain the canonical support of stored atoms;
2. narrow queries to sound candidate sets;
3. remain unchanged under duplicate adds or non-last removes.

It is *not* the whole semantic carrier of a MeTTa space. Bag-level multiplicity remains a separate refinement layer.

The concrete selector development in `CandidateArchitecture.lean` now proves:

1. `concrete_twoPhase_eq_direct`: trie candidate selection followed by native exact matching agrees with direct querying;
2. `trieCandidates_subset_support`: trie candidates are phantom-free;
3. `realPrefixSelector_sound`: real subtrie descent under an encoded prefix is sound for candidate narrowing;
4. add/remove zero-crossing theorems: duplicate adds and non-last removes do not change trie support, while first adds and last removes do.

Positive example. If a space stores two copies of the same atom, PathMap should still hold one canonical support element while a counted-space refinement records multiplicity 2.

Negative example. Encoding multiplicity by inventing different trie keys for duplicates would destroy the structural-sharing and canonical-support story that makes PathMap valuable in the first place.

This division of labor is useful for MORK as well. The trie should perform prefix descent and support-level candidate narrowing; exact variable-sensitive matching belongs in the native matcher that consumes those candidates. `PathMapMatcherInstance.lean` now makes that bridge explicit by instantiating the abstract two-phase theorem with the concrete HE matcher. The broader binder-hygiene and cache-correctness obligations remain in the companion backend note rather than in this trie-focused paper.

13 MeTTaIL Bridge

The MeTTaIL bridge (`MeTTaILBridge.lean` together with `AuthoredContextBridge.lean` and `CollectionBridge.lean`) connects MORK’s execution model to MeTTaIL’s least reduction generated by authored rules (`ContextualStep.Step`).

The mutual recursive function `morkPatternToAtom` translates MeTTaIL patterns to MORK atoms. The predicate `morkTranslatable` identifies soundly translatable patterns (no beta-redex, no rest-variables).

13.1 Zero-Premise Bridge

Theorem 13.1 (`stepAt_compiles_to_mork_fire`). *If a bounded MeTTaIL authored contextual derivation relates a source to a target and both translations are ground, then a rule compiled from exactly that derivation fires in MORK and replaces the translated source by the translated target.*

13.2 Multi-Premise Bridge via PremiseChain

MeTTaIL rules may have *premises*: side-conditions that must be satisfied for the rule to fire. The two premise types relevant to MORK are `relationQuery` (workspace lookup) and `freshness` (variable-distinctness guard).

A `PremiseChain` is an inductive proof that a list of premises can be satisfied by workspace atoms:

Listing 13: `PremiseChain` inductive type.

```
inductive PremiseChain (relEnv) (lang) (s : Space)
  : Bindings -> List Premise -> List Atom -> Bindings -> Prop where
| nil : PremiseChain relEnv lang s bs [] [] bs
| cons : (prem : Premise) -> (a : Atom) ->
  a ∈ s ->
  matchAtom (σ0) (premiseToAtomTotal prem) a = some σ1 ->
  b1 ∈ premiseStepWithEnv relEnv lang b0 prem ->
  PremiseChain relEnv lang s b1 prems witnesses bf ->
  PremiseChain relEnv lang s b0 (prem :: prems)
    (a :: witnesses) bf
| guard : (prem : Premise) ->
  premiseToSourceFactor prem = none ->
  b0 ∈ premiseStepWithEnv relEnv lang b0 prem ->
  PremiseChain relEnv lang s b0 prems witnesses bf ->
  PremiseChain relEnv lang s b0 (prem :: prems) witnesses bf
```

The `cons` constructor handles `relationQuery` premises (consuming a workspace atom as witness), while `guard` handles `freshness` premises (checking a substitution-level condition without consuming workspace atoms).

Theorem 13.2 (`languageStep_implies_mork_fireSourceRule`). *If an authored `ContextualStep.Step` fires in a language whose rules have only non-contextual, source-translatable premises, and a `PremiseChain` witness exists in the workspace, then the corresponding `SourceExecRule` compiled from that language fires in `fireSourceRule`.*

13.3 Freshness Guards and the Extended Bridge

`SourceGuard.freshness` encodes the condition that a variable does not occur free in a substituted term. The correspondence theorem bridges MeTTaIL’s sequential freshness checking to MORK’s guard filtering:

Theorem 13.3 (`freshness_premise_correspond`). *If MeTTaIL’s freshness premise succeeds at bindings β , then `matchSourceGuard (bindingsToSubst  $\beta$ ) g = true` for the translated guard g .*

The extended bridge handles premise lists mixing `relationQuery` and `freshness`:

Theorem 13.4 (`languageStep_implies_mork_fireSourceRuleExt`). *If an authored `ContextualStep.Step` fires in a language whose rules have only non-contextual ext-translatable premises (`allPremisesTranslatableExt`), a `PremiseChain` witness exists, and the extracted guards pass at the final substitution, then the translated `SourceExecRule` (with guards) fires in `fireSourceRule`.*

Design note. MeTTaIL checks freshness guards *sequentially* at the point each guard fires in the premise chain, while MORK’s `fireSourceRule` checks guards at the *final* substitution. These semantics differ in general: extending a substitution can break a previously-satisfied freshness condition. The bridge therefore accepts guard satisfaction at the final substitution as an explicit hypothesis. For typical rules where guard variables are fully bound by earlier factor-matching premises (as in all concrete PLN rules below), this hypothesis is trivially dischargeable.

13.4 Concrete PLN Rules as GSLT Routing Witnesses

`PLNRules.lean` (378 lines, 23 theorems) instantiates the bridge for concrete PLN inference rules, providing end-to-end GSLT routing witnesses:

Rule	Premises	Conclusion	Guards
Transitivity	$\text{Inh}(A,B), \text{Inh}(B,C)$	$\text{Inh}(A,C)$	—
Modus ponens	$\text{Impl}(P,Q), \text{Eval}(P)$	$\text{Eval}(Q)$	—
$\text{Inh} \rightarrow \text{Similarity}$	$\text{Inh}(A,B)$	$\text{Sim}(A,B)$	—
Abduction	$\text{Inh}(A,B), \text{Inh}(C,B)$	$\text{Sim}(A,C)$	—
Guarded trans.	$\text{Inh}(A,B), \text{Inh}(B,C)$	$\text{Inh}(A,C)$	$A\#C$
Double-guarded	$\text{Inh}(A,B), \text{Inh}(B,C)$	$\text{Inh}(A,C)$	$B\#A, B\#C$

All translatability proofs are by `decide`. Two language definitions package these rules: `plnPremiseLanguageDef` (3 unguarded rules, using the strict bridge) and `plnGuardedPremiseLanguageDef` (3 guarded/mixed rules, using the ext bridge). Concrete bridge instantiation theorems (`pln_step_implies_mork_fireSourceRule` and `pln_guarded_step_implies_mork_fireSourceRule`) demonstrate the full chain from the least authored contextual relation to `fireSourceRule` for each language definition.

Remark 13.5 (Honest Scope). The multi-premise and extended bridges handle flat rules where the LHS is a free variable and all premises are non-contextual. Authored contextual premises are handled separately by compiling their bounded derivations through the context bridge (§13.5). Beta-redex patterns and non-ground results remain outside the formalized boundary.

13.5 Authored Context and Collection-Shaped Lens Bridge

`AuthoredContextBridge.lean`, `CollectionBridge.lean`, and `AtomZipper.lean` prove that an authored one-hole context corresponds to a MORK-side update through a principled zipper/lens factorization. Collection shape by itself grants no reduction authority: the language must contain a rewrite rule whose `Premise.congruence` explicitly authorizes the frame.

The bridge is organized in layers:

Layers A–B: Structural infrastructure. `morkPatternToAtomList` commutes with `List.set` (Layer A), and ground atoms match themselves under any substitution (Layer B):

```
theorem morkPatternToAtomList_set (elems : List ILP)
  (i : Nat) (q' : ILP) :
  morkPatternToAtomList (elems.set i q') =
  (morkPatternToAtomList elems).set i
  (morkPatternToAtom q')

theorem matchAtom_ground_self (σ : Subst) (a : Atom)
  (hg : isGroundAtom a = true) :
  matchAtom σ a a = some σ
```

Layer C: Structural context translation. The derivative `OneHoleContext` is translated to a MORK `AtomContext`. Rebuilding commutes with translation, and an `AuthoredContextFrame` retains the rule and congruence-premise witness that authorize the frame.

Layer D (primary): Huet zipper and lens relation. `AtomZipper.lean` formalizes a concrete Huet zipper over `Atom`, following MORK’s Rust `ExprZipper`:

```
structure AtomCrumb where
  left  : List Atom  -- reversed left siblings
  right : List Atom  -- right siblings

abbrev AtomContext := List AtomCrumb
structure AtomZipper where
  focus : Atom
  ctx   : AtomContext
```

Navigation (`descendExprChild?`, `ascend?`), path-indexed focus (`focusAtPath?`), and path-indexed replacement (`replaceAtPath?`) are proven to compose correctly. A `LensRel` captures the get/put/put-put factorization:

```
def LensRel (whole part newPart newWhole : Atom)
  : Prop :=
  ∃ z : AtomZipper,
  rebuild z = whole ∧ z.focus = part ∧
  rebuild (replaceFocus z newPart) = newWhole
```

Three classical lens laws are proven (0 sorries): `lensRel_get_put` (replace focus with itself is identity), `lensRel_put_get` (focus of replacement equals new value), and `lensRel_put_put` (consecutive replacements compose).

The collection specialization `collectionContext_lensRel` proves that replacing element i in a `.expression (ctSym :: elems)` satisfies `LensRel`, connecting the zipper decomposition to the structural derivative. The stronger `authoredCollectionContext_lensRel` additionally retains the authorizing language-rule witness.

The canonical authored-context bridge combines the structural and semantic layers:

Theorem 13.6 (`authoredContextFrame_lensRel`). *An authored one-hole context has an exact `LensRel` realization after translation to MORK, and the result retains both the authorizing language rule and its contextual-premise witness.*

Layer E: Exact contextual compilation. `compiledContextRules` contains exactly the targets emitted by the bounded authored contextual compiler, and `stepAt_compiles_to_mork_fire` proves that every such derivation performs the corresponding translated update.

Layer F: Source-aware contextual compilation. `compiledContextSourceRules` similarly contains exactly the bounded authored targets as source-aware rules, and `stepAt_compiles_to_mork_sourceRule` proves their workspace update.

Source-rule provenance boundary. The rules in `compiledContextRules` and `compiledContextSourceRules` are derived from accepted contextual reductions; they are not appended as independent language rules. The ordinary `languageDefToSourceExecRules` bridge remains restricted to authored rules with non-contextual premises. This keeps contextual authority in the source language while allowing the backend to realize an accepted contextual step as a whole-atom replacement.

13.6 Compound Pattern Round-Trips

The space-effect rules for `add-atom` and `remove-atom` (`SpaceEffectFragment.lean`, 193 lines) use compound command patterns like `(add-atom &self $x)`. Previously, the correctness of matching these patterns against concrete command expressions was an MM2 compiler invariant expressed as a hypothesis.

Using the `MatchAtomRel` relational specification (`MatchSpec.lean`), we now construct explicit derivations (`expr_cons / symbol / var_fresh`) and apply `matchAtom_complete` to obtain proven round-trip theorems:

Theorem 13.7 (`matchAtom_addAtomCmdAtom`). *For any MeTTaIL pattern p , $\text{matchAtom } [] \text{ addAtomCmdAtom } (\text{morkPatternToAtom } (. \text{apply } "add-atom" [.\text{apply } "\&self" [], p)]) = \text{some } [("x", \text{morkPatternToAtom } p)]$.*

These are then lifted to source-rule firing level:

Theorem 13.8 (`addAtom_fireSourceRule_mem`). *When the command atom is in the workspace, `addAtomSourceExecRule` fires and produces the `applySinks` result with binding $"x" \mapsto \text{morkPatternToAtom } p$.*

Symmetric theorems hold for `remove-atom`.

13.7 Extended Bridge: Beyond morkTranslatable

The base MeTTaIL bridge requires `morkTranslatable r.right` (no `.subst` nodes, no rest-variable collections). The extended bridge (`ExtendedBridge.lean`, 83 lines, plus supporting infrastructure in `MeTTaILBridge.lean` and `Substitution.lean`) lifts this restriction by proving that `applyBindings` always normalizes these constructs away.

Pattern normalization. A mutual recursive function `normalize / normalizeList` eliminates all `.subst` nodes by evaluating them via `openBVar`. The key property:

Theorem 13.9 (`noExplicitSubst_normalize`). *For all patterns p , `noExplicitSubst (normalize p) = true`.*

morkTranslatable preservation under openBVar. The theorem `morkTranslatable_openBVar` proves that substituting a translatable term into a translatable pattern preserves translatability. This is proven by structural induction on `Pattern.inductionOn`, with the `.subst` case vacuously true (no translatable `.subst` exists) and the `.collection` case requiring a match on the rest field.

Core theorem.

Theorem 13.10 (`morkTranslatable_applyBindings`). *For any bindings bs and pattern rhs , if all values in bs are `morkTranslatable`, then `morkTranslatable (applyBindings bs rhs) = true`.*

Note: no `morkTranslatable rhs` hypothesis is needed. `applyBindings` eliminates `.subst` (via `openBVar`) and resolves rest-variables (via lookup and splice with `rest = none`), so the output is always in the `morkTranslatable` fragment regardless of the input structure.

The proof handles each `Pattern` constructor: `.fvar  $x$` looks up in bindings (translatable by hypothesis or identity); `.subst` recurses on body and replacement, then applies `morkTranslatable_openBVar`; `.collection` maps elements (IH), extracts rest-variable elements from bindings (translatable since the bound collection value is translatable), and appends via `morkTranslatableList_append`.

Extended fire theorems.

Theorem 13.11 (`declReduces_extended_mork_fire`). *A MeTTaIL rewrite step fires via an ad-hoc exec rule even when the rule's RHS uses `.subst` or rest-variables. The witness uses `collectionReplaceRule` with groundness of both old and new atoms.*

A source-rule variant `declReduces_extended_mork_sourceRuleFire` provides the same guarantee at the `SourceExecRule / fireSourceRule` level.

14 MQ-Calculus Bridge

The file `MORKCommBridge.lean` formalizes the correspondence between MORK's binary fold steps and the MQ-calculus `CommReduction` rule.

Theorem 14.1 (`mork_fold_is_comm`). *For each binary fold outcome, there exists an MQ `CommReduction` with the matching branch selection.*

Theorem 14.2 (`mork_fold_both_outcomes_exist`). *Both outcomes are constructively possible for any binary fold step: non-determinism is structural.*

Theorem 14.3 (`mork_mq_nondeterminism_corresponds`). *MORK's binary fold non-determinism (both sub-results reachable) is equivalent to MQ's `comm_both_outcomes`.*

14.1 N-ary Generalization

The binary case is a special case of N-ary fold non-determinism:

Theorem 14.4 (`nary_fold_all_outcomes_exist`). *For any fold step with N sub-results and any index $k < N$, there exists a fold step selecting sub-result k as the assembled output.*

Theorem 14.5 (`binary_subResult0_is_nary`). *Binary `FoldPicksSubResult` for sub-result 0 is equivalent to `NaryFoldPicksSubResult` at index 0.*

Theorem 14.6 (`nary_fold_has_binary_comm`). *Any N-ary fold with ≥ 2 sub-results has both `COMM` outcomes available.*

Seven canary theorems protect these invariants.

15 Z3 Oracle Semantics

`Z3Oracle.lean` (347 lines, 27 theorems) formalizes the oracle seam of the MORK runtime, connecting abstract oracle queries to concrete Z3 behavior. The Z3 subprocess interaction in the Rust runtime follows a three-step protocol: (i) sink-side: buffer SMT-LIB statements and write to Z3 stdin; (ii) source-side: send (`check-sat`) and (`get-model`), parse the response; (iii) pattern-match against the parsed model using the existing `matchOneInSpace` infrastructure.

Definition 15.1 (`OracleEnv`). An oracle environment is a pure function `OracleQuery` $\rightarrow$ `OracleResponse` abstracting the Z3 subprocess lifecycle.

The key bridge function `oraclePayloadSpace` converts an oracle response’s payload atoms into a `Space` (`Finset Atom`), enabling reuse of the existing pattern matching infrastructure:

Theorem 15.2 (`oracleMatchPattern_sound`). *Oracle pattern matching agrees with `matchOneInSpace` on the payload space: a match against the oracle response succeeds iff it succeeds against the flattened space.*

Oracle queries are routed by resource type (`z3/act`):

Theorem 15.3 (`routeOracleQuery_z3_routes`). *A query with resource tag "z3" routes to the Z3 oracle environment.*

A mock Z3 environment (`mockZ3Env`) with 7 conformance tests validates the architecture against the MORK Rust test `sink_z3_basic`.

Remark 15.4 (Honest scope). This file does NOT model the Z3 subprocess lifecycle (spawn/kill), SMT-LIB parsing, or PathMap zipper internals. The oracle is a pure function parameter—the implementation gap is the Lean–Rust FFI for subprocess I/O.

16 Invertible Compat-Head Fragment

`InvertibleHead.lean` (2,766 lines, 61 theorems) is the largest module in the formalization. It formalizes the compilation of PeTTa’s *evaluation-during-matching* (`matchHeadArgWithEval` in the PeTTa Dispatch module) to standard MORK three-phase execution.

16.1 Problem

PeTTa’s pattern matcher can *evaluate* subterms during matching: when matching $(f \ \$x)$ against a query, PeTTa calls the interpreter to evaluate f and matches the result against the query. MORK’s `matchAtom` is purely structural—it cannot evaluate during matching.

16.2 Key Insight

For a significant subfragment—head functions that are *pure, structurally recursive, and produce constructor-wrapped outputs*—the evaluation-during-matching can be compiled to standard MORK/MM2 by *reversing* the computation: instead of evaluating $f(x)$ forward and matching, we reverse-match the query argument against f ’s output structure to recover x .

16.3 Formalization

The file defines:

Listing 14: Core definitions (`InvertibleHead.lean`).

```
structure HeadEquation where
  funcName : String
  lhsArgs  : List Atom      -- LHS patterns
  rhs      : Atom           -- RHS expression

structure HeadFuncDef where
  funcName : String
  equations : List HeadEquation
  consistent : ∀ eq ∈ equations, eq.funcName = funcName

structure InvertibleHead (def_ : HeadFuncDef) : Prop where
  hasBase      : ∃ eq ∈ def_.equations, eq.isBase
  varsCovered  : ... -- RHS vars ⊆ LHS vars
  basesAreCtorApps : ... -- base RHS = ctor application
  stepsWrapRecursion : ... -- step RHS wraps one recursive call
  lhsVarsRecoverableBase : ... -- LHS vars recoverable from RHS
  lhsVarsRecoverableStep : ... -- LHS vars recoverable from shell
```

The compilation proceeds in layers:

1. **Forward evaluation** (`forwardEval`): fuel-bounded recursive evaluation following the defining equations.
2. **Reverse matching**: `reverseMatchBase` decomposes a constructor application to recover LHS variables; `reverseMatchStep` strips the outer constructor shell and extracts the recursive sub-query.
3. **Staged compilation**: the three-phase UNFOLD/BASE/FOLD architecture maps to: UNFOLD = strip outer shell and emit recursive sub-query + wait token; BASE = reverse-match base case; FOLD = reassemble original input from sub-result.

The main soundness chain:

Theorem 16.1 (`reverseEval_inverts_forwardEval`). *For a deterministic invertible fragment: if `forwardEval` on a ground input produces output `out`, then `reverseEval` on `out` recovers the original input.*

Theorem 16.2 (`staged_inverse_sound`). *Every successful forward evaluation gives rise to a `StagedInverseTrace`—a structured proof object recording the UNFOLD/BASE/FOLD decomposition that recovers the original input.*

Theorem 16.3 (`staged_inverse_exec_sound`). *The staged trace refines to a concrete scheduler-facing `StagedInverseExecTrace`, provided explicit singleton witnesses are supplied for each base/unfold/fold node.*

Remark 16.4 (Honest scope). The bridge to `matchHeadArgWithEval` (PeTTa Dispatch module) is not yet formalized. The current scope covers the *fragment boundary* and *compilation scheme soundness*; the connection to the PeTTa surface language is future work. Non-injective head functions are excluded by the `InvertibleHead` predicate.

17 Direct-Exec Surface

`DirectExecSurface.lean` (234 lines) packages the scheduler-level exactness theorems (Tasks A/B/C from `ThreePhaseRefinement.lean`) into a single reusable `SchedulerSeam` and composes the first real PeTTa fragment through it.

17.1 Scheduler Seam

Definition 17.1 (`SchedulerSeam`). A coherent family of re-exported theorems:

- `orderAgreement`: scheduler byte-key order agrees with shortlex on (priority, name).
- `baseExactness` / `unfoldExactness` / `foldExactness`: each phase’s scheduler step equals the corresponding phase operation.
- `sourceRelation`: `fireExecFact = source – exec atom`.
- `progress`: if `selectNextExec` returns `some`, then `cWorkQueueStep` succeeds.

17.2 PeTTa Fragment Bridge

The key theorem lands a PeTTa `eval/evalc` step on the MM2 scheduler:

Theorem 17.2 (`pettaEvalStep_schedulerFires`). *A PeTTa `evalStep` that translates to a MORK `exec` rule fires through the scheduler seam.*

Remark 17.3 (Honest scope). This file explicitly lists what is NOT covered: `add-atom/remove-atom` (MORK sinks ready, MeTTaIL-level rule missing), collection-pattern matching (excluded by `morkTranslatable`), and beta-reduction (`.subst` nodes rejected).

18 Conformance Testing

The conformance test suite (`Conformance.lean`, 1,694 lines) provides 13 kernel-checked conformance theorems verified against the MORK CLI (`mork run`), plus bidirectional match-pattern and fire-rule correspondence theorems between the computable and spec levels.

18.1 Computable Reference Evaluator

The spec-level `fireRule` is `noncomputable` (uses `Finset.toList`). The computable reference evaluator mirrors these operations over `List Atom`:

- `cmatchAtom` / `cmatchAtomList`: mutual recursive matcher that extends `matchAtom` to handle `.expression` patterns by element-wise matching.
- `cmatchPattern`: conjunctive matching over a list space.
- `capplySinks`: sink application on list spaces (handles `add`, `remove`, and `head` with explicit membership check for idempotence).
- `cfireRule`: end-to-end rule firing.

All operations reduce by `rfl` in the Lean kernel.

18.2 Test Fixtures

Each test is defined as an `ExecRule` applied to a concrete space, with a theorem asserting that `cfireRule` produces the expected output. The corresponding `.mm2` file is run through `mork run` to obtain the ground-truth output.

Test 8 exercises multi-step exhaustive execution: the rule fires three times, consuming each `(task x)` atom and producing `(done x)`. The formalization proves each step individually via `rfl` (the fixpoint function does not reduce definitionally, so a chain of one-step equalities is used instead).

18.3 Expression Matching

The spec-level `matchAtom` returns `none` on `.expression` patterns. The computable evaluator's `cmatchAtom` extends this via mutual recursion:

Listing 15: Expression matching (`Conformance.lean`).

```
mutual
def cmatchAtom (sig : Subst) (pat conc : Atom)
  : Option Subst :=
  match pat, conc with
  | .expression ps, .expression cs =>
    cmatchAtomList sig ps cs
  | .var v, a => ... -- same as matchAtom
  | ...
def cmatchAtomList (sig : Subst) (pats concs
  : List Atom) : Option Subst :=
  match pats, concs with
  | [], [] => some sig
  | p :: ps, c :: cs =>
    match cmatchAtom sig p c with
    | some sig' => cmatchAtomList sig' ps cs
    | none => none
  | _, _ => none
end
```

This enables conformance tests with real MM2 S-expressions (tests 1, 3–8).

19 Coverage and Open Gaps

Table 4 summarizes the boundary between formalized and unformalized aspects.

19.1 Notable Restrictions

1. **Expression matching.** The spec-level `matchAtom` now handles expression patterns (via mutual recursion with `matchAtomList`). The biconditional `matchAtom_iff` covers the `morkTranslatable` fragment (excluding `.subst` and rest-variable `.collection`). However, `morkTranslatable_applyBindings` (§13.7) proves that `applyBindings` eliminates both constructs, so rule RHS patterns containing `.subst` or rest-variables are fully handled at the bridge level.
2. **Coreferential matching.** MORK’s Rust backend uses de Bruijn-style coreferential matching with byte capture (`$ps`, `$cs`). The formalization uses first-order variable binding.
3. **Aggregation sinks.** The formalization captures `add`, `remove`, `head`, and fold-level `count/sum/selectFirst/s`. The Rust backend’s remaining 11 per-match sink types (`min`, `max`, etc.) are not formalized.
4. **Aggregator enforcement.** The `AggregatorConsistent` predicate (re-exported via `ExecutionBoundary.ag`) now connects fold-assembled results to aggregator semantics. Its use as a runtime invariant is not yet enforced.
5. **Computability.** The spec-level space operations are `noncomputable` (`Finset.toList`). The computable reference evaluator provides a kernel-reducible alternative for conformance testing. The scheduler correspondence chain (§6.4) bridges the gap: under `WorkQueueInvariant`, the computable scheduler agrees with the spec through arbitrary bounded-run horizons.
6. **FTrie commutativity.** FTrie join is left-biased ($v_1 <|> v_2 = v_1$), so `JoinComm` does not hold in general. For $V = \text{Unit}$ (MORK’s use case), commutativity is trivial.
7. **MQ bridge witnesses.** The MQ-MORK bridge uses `Process.MQNil` witnesses—the correspondence is structural but the witnesses carry no information about the actual MORK processes.
8. **Metatheory.** Termination for ground removing programs and confluence for deterministic rule sets are not yet formalized.
9. **Native path-focused contextual execution.** The zipper/lens layer provides the structural semantics for sub-expression access, and bounded authored contextual steps already compile exactly to source-aware whole-atom replacement rules. A future backend optimization could encode the same accepted step with path-aware source factors (`SourceFactor.focus`) and sink actions (`SinkAction.replaceAtPath`); it must prove agreement with the authored contextual relation rather than add new reduction authority.
10. **PathMap Rust coverage.** The PathMap Lean formalization covers the algebraic semantics (`AlgebraicResult`, lattice/quantale hierarchy) and adds OSLF/evidence bridges not present in Rust. However, significant Rust subsystems are not formalized: `ZipperWriting` mutation operations (444 Rust LOC), paths serialization (19,000 Rust LOC), catamorphism/anamorphism morphisms (800 Rust LOC), the Arena-Compact on-disk format, merkleization/deduplication, product/overlay zipper composition, and concrete node type selection (`Dense/Line/Tiny`). The Lean formalization targets *algebraic correctness and semantic soundness*, not full Rust implementation verification.
11. **Invertible head scope.** The `InvertibleHead` predicate (§16) covers pure, structurally recursive functions with constructor-wrapped outputs and injective base cases. Non-injective heads, multi-argument recursive calls, and the bridge to PeTTa’s `matchHeadArgWithEval` are not yet formalized.

12. **PathMap OSLF completeness.** The NTT completeness theorems (§10) are exact biconditionals parameterized by a `relEnv : RelationEnv`. The connection from a concrete PathMap store to a `RelationEnv` satisfying the required premises is handled by `RelationBridge.lean`, but automatic instantiation (given a concrete `Finset`) is not yet provided.
13. **Zipper complexity.** `ZipperComplexity` (§9) formalizes structural depth bounds (path length grows by at most k). Actual time-complexity proofs require a cost model not available in Lean.

20 Execution Boundary

`ExecutionBoundary.lean` (510 lines) provides a thin re-export surface packaging the proven `morkTranslatable` execution boundary, the scheduler correspondence chain, the extended bridge infrastructure, the zipper/lens congruence infrastructure (`AtomZipper`, `LensRel`, lens laws), the collection congruence bridge, and the extended `.subst`/rest-variable bridge. The predicate `morkTranslatable` identifies the MeTTaIL pattern fragment for which substitution commutation holds. However, the extended bridge (§13.7) proves that patterns *outside* this fragment (containing `.subst` or rest-variable collections) are still handled: `applyBindings` normalizes them into the fragment, and ad-hoc replace rules provide execution witnesses.

This file re-exports key definitions (`morkTranslatable`, `morkPatternToAtom`, pattern-to-space translation, rewrite-rule translation), scheduler correspondence theorems (`cFireExecFact_toFinset`, `cWorkQueueStep_toFinset`, `cWorkQueueRunN_toFinset`, `WorkQueueInvariant`), source guard infrastructure (`SourceGuard`, `matchSourceGuard`, `matchSourceGuards`, `isAtomFresh`, `freshness_premise_corresp`), extended bridge theorems (`premiseChainMatchFactorsExt`, `multiPremiseExtBridge`, `fullExtBridge`, `languageDefSourceRulesExt`), zipper/lens infrastructure (`AtomZipper`, `AtomCrumb`, `LensRel`, and the lens laws), authored-context bridge theorems (`authoredContextLensBridge`, `contextualRuleCompilation`, `contextualSourceRuleCompilation`, `contextualStepFires`, `contextualStepSourceFires`, `collectionContext`, and `authoredCollectionContextLensBridge`), and the extended bridge (`patternNormalize`, `normalizeSubstFresh`, `openBVarPreservesTranslatable`, `applyBindingsPreservesTranslatable`, `extendedExecFire`, `extendedSourceRuleFire`) as public abbrevs, providing a stable API surface for downstream consumers.

21 Runtime-Kernel Packaging

Two companion files above the MORK layer package the current proved execution seams into a backend-neutral runtime-kernel object that the `ElaboratedCore` elaboration layer can target without importing scheduler internals.

21.1 Effect-Class Bridge (`RuntimeKernel.lean`)

`RuntimeKernel.lean` (211 lines) bridges the `RuntimeKernelClass` enum (from `ElaboratedCoreBase.lean`) to the `EffectClass` hierarchy (from `mettail-core/EffectSafety.lean`).

	<code>.ruleExec</code>	$\mapsto$ <code>pureStructural</code>
	<code>.query</code>	$\mapsto$ <code>readOnlyLookup</code>
Definition 21.1 (<code>RuntimeKernelClass.effectClass</code>).	<code>.spaceEffect</code>	$\mapsto$ <code>writesState</code>
	<code>.oracle</code>	$\mapsto$ <code>oracleIO</code>
	<code>.metaPhase</code>	$\mapsto$ <code>pureStructural</code>

A `ClassifiedFragment` pairs a kernel class with its effect class, resource class, and backend name. Three canonical fragments package the current MORK/MM2-backed witnesses:

- `execFragment`: `eval/evalc` → `pureStructural` on `defaultAtomSpace`
- `queryFragment`: `match/get-atoms` → `readOnlyLookup` on `defaultAtomSpace`
- `spaceEffectFragment`: `add-atom/remove-atom` → `writesState` on `defaultAtomSpace`

A `RuntimeKernelPackage` bundles the `MeTTaRuntimeKernelTriad` with these classified fragments. The canonical instance `morkRuntimeKernelPackage` uses all three.

Memoization safety is kernel-checked: `exec` and `query` fragments support both scalar and outcome-set memoization; `spaceEffect` supports neither (by `decide`). All theorems have zero axioms.

21.2 Resource Layer (`RuntimeResource.lean`)

`RuntimeResource.lean` (234 lines) adds an explicit resource layer classifying which resources the current MORK/MM2 backend supports.

Definition 21.2 (`ResourceDescriptor`). A descriptor carries: resource class, supported kernel classes, effect envelope, backend name, and a Boolean `hasProvedExecSeam`.

Four descriptors are defined, aligned with the MORK Rust implementation:

The `solverResource` and `externalResource` descriptors acknowledge that MORK’s Rust runtime actively uses Z3 subprocesses (`ResourceRequest::Z3` in `space.rs`) and ACT file-backed external datasets. The Z3 oracle semantics are now partially formalized (`Z3Oracle.lean`, §15): query routing, payload-to-space conversion, and pattern matching are proved, but the subprocess lifecycle and SMT-LIB parsing remain unformalized. ACT external files are not yet formalized. Named atomspaces are not supported by MORK’s single-space-per-instance architecture.

A `ResourceProfile` pairs a `RuntimeKernelPackage` with its resource descriptors and identifies the primary resource. The canonical `morkResourceProfile` uses `defaultAtomSpace` as the primary resource (the only one with a proved execution seam).

All classification theorems (19 in `RuntimeKernel.lean`, 16 in `RuntimeResource.lean`) are proved by `rfl` or `decide` with axioms at most [`propext`, `Classical.choice`, `Quot.sound`].

Future Work: Multiset Semantics via Semiring Algebra

`PathMap` implements set semantics: inserting the same expression twice yields one copy, and algebraic operations collapse duplicates. This is by design for MORK/MM2, where deterministic execution requires deduplication. However, MeTTa has *multiset* semantics: adding the same atom twice creates two copies, and queries return one result per copy.

Problem. When `PathMap` backs ordinary MeTTa spaces (not MORK-managed MM2 spaces), the set semantics create a semantic mismatch. CeTTa currently works around this via a parallel index mapping serialized atoms to row-ID vectors, but this index does not compose with `PathMap`’s algebraic operations.

Approach A: Embedded count. Encode multiplicity in a CeTTa-owned counted key formed by

$$\text{atom_expr_bytes} \parallel \text{count_expr_bytes}$$

where the count remains an ordinary serialized number expression. This keeps the atom bytes at the front of the key, preserving prefix scans for PathMap narrowing. Using the ordinary pair expression (`$atom $count`) directly would *not* preserve the atom prefix under the current MORK expr serialization, because the outer arity tag appears before the atom payload. On insert, look up the existing count and re-insert with the incremented value. On query, decode the count and emit results accordingly.

- Works with current PathMap API.
- Count rewrites remain a CeTTa-owned concern; algebra ops still require special handling.

Approach B: PathMap<CountValue> with semiring algebra. A cleaner architecture uses PathMap<V> with `V = CountValue` (a newtype over `u32`). This requires:

1. **Semiring Lattice instance for CountValue.** The current numeric `Lattice` implementations in `ring.rs` are placeholders that ignore the “other” operand. A proper multiset semiring requires:
 - `pjoin` (union): $\text{count}_a + \text{count}_b$
 - `pmeet` (intersection): $\min(\text{count}_a, \text{count}_b)$
 - `psubtract` (difference): $\max(0, \text{count}_a - \text{count}_b)$
2. **Query-level count multiplication for conjunctions.** Storage gets cleaner under this model, but query evaluation must still multiply counts across matched factors.

Note that `join_val_at` is a convenience, not a strict blocker: PathMap already supports in-place insert-or-mutate via `write_zipper_at_path` (`trie_map.rs:450`).

Theoretical foundation. Multiset (bag) semantics over annotated relations are characterized by *provenance semirings* [9]. Query evaluation is parameterized over a commutative semiring $(\mathbb{K}, +, \times, 0, 1)$; for counting multiplicities, the natural numbers $(\mathbb{N}, +, \times, 0, 1)$ suffice. Join operations correspond to multiplication (Cartesian product count), union to addition, and projection sums multiplicities over projected-away witnesses.

See also Mumick et al. [10] on bag semantics in Datalog and Abiteboul–Hull–Vianu [11] on bag relational algebra.

Listing 16: Semiring Lattice for multiset counts (Rust pseudocode).

Implementation sketch.

```
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
struct CountValue(u32);

impl Lattice for CountValue {
    fn pjoin(&self, other: &Self) -> AlgebraicResult<Self> {
        // Union = sum of counts
        AlgebraicResult::Element(CountValue(
            self.0.saturating_add(other.0)))
    }
    fn pmeet(&self, other: &Self) -> AlgebraicResult<Self> {
        // Intersection = min of counts
        let m = self.0.min(other.0);
        if m == 0 { AlgebraicResult::None }
        else { AlgebraicResult::Element(CountValue(m)) }
    }
}
```

```

impl DistributiveLattice for CountValue {
  fn psubtract(&self, other: &Self) -> AlgebraicResult<Self> {
    // Difference = saturating subtraction
    let d = self.0.saturating_sub(other.0);
    if d == 0 { AlgebraicResult::None }
    else { AlgebraicResult::Element(CountValue(d)) }
  }
}

```

Benefits.

- Multiset semantics compose with all PathMap algebra operations (no parallel index synchronization).
- Conjunction multiplicity computes correctly: product of factor counts, per semiring multiplication.
- Clean separation: MORK/MM2 continues using `PathMap<()>` (set semantics), while MeTTa spaces use `PathMap<CountValue>` (multiset semantics).

Immediate CeTTa-owned path. Approach A is already enough to give CeTTa a mainline-compatible prototype for `new-space pathmap`: one counted key per distinct atom, local insert/remove-one rewrite, local query rematch, and local conjunction logic that multiplies factor counts. In that lane, duplicate row ids become a derived compatibility view rather than stored truth, and the helper `query_multi_with_factor_exprs` is no longer structurally required. If CeTTa later chooses to store multiple key families in one shared PathMap, an explicit discriminator byte can be added at that point; the simplest fresh-space prototype does not require one.

Formalization opportunity. The Lean formalization could extend `PathMapQuantale` with a `CountValue` instance and prove semiring laws distribute over the `FTrie` operations. The existing `AlgebraicResult` infrastructure handles the zero-count $\rightarrow$ deletion case via the `.none` variant. This would provide mechanically-verified multiset semantics for PathMap-backed spaces within the Mettapedia codebase.

22 Storage vs Computation Cost Analysis

This section formalizes when PathMap storage should be preferred over on-demand computation. The theory is mechanized in Lean 4 with zero `sorry` statements.

22.1 Crossover Theorem

For a function $f : \alpha \rightarrow \beta$ with finite domain $|\alpha| = n$:

- **Storage cost:** $S = n \cdot \text{entrySize}$ (upfront) plus $O(1)$ per lookup
- **Computation cost:** $O(T)$ per evaluation (no upfront cost)

Theorem (`storage_better_iff`): Storage beats computation after q queries if and only if

$$S < q \cdot (T - 1)$$

Equivalently, storage wins when $q > S/(T - 1)$.

```

theorem storage_better_iff (storageCost computeTime queries : Nat)
  (h : computeTime > 1) :
  totalCost_stored storageCost queries <
  totalCost_compute computeTime queries ↔
  storageCost < queries * (computeTime - 1)

```

22.2 Reverse Query Advantage

Theorem (reverse_stored_wins): For reverse queries (given output, find inputs), stored tables *always* beat enumeration:

$$|\{x : f(x) = y\}| \leq |\alpha|$$

Proof: `Finset.card_filter_le`.

22.3 Concrete Arithmetic Analysis

For n -bit integers with domain size $N = 2^n$:

Operation	Compute/Query	Table Size	Crossover
Addition	$O(n)$	$N^2 \cdot (n + 1)$ bits	$\approx N^2$ queries
Multiplication	$O(n^2)$	$N^2 \cdot 2n$ bits	$\approx N^2/n$ queries
Division	$O(n^2)$	$N^2 \cdot n$ bits	$\approx N^2/n$ queries

8-bit arithmetic ($n = 8$, $N = 256$).

- Addition table: $256^2 \times 9 \approx 590$ KB; compute: 8 ops/query
- Multiplication table: $256^2 \times 16 \approx 1$ MB; compute: 64 ops/query
- **Recommendation:** Storage is practical; multiplication benefits most

16-bit arithmetic ($n = 16$, $N = 65536$).

- Addition table: $65536^2 \times 17 \approx 73$ GB — *impractical*
- Multiplication table: $65536^2 \times 32 \approx 137$ GB — *impractical*
- **Recommendation:** Always compute on demand

22.4 Trie vs Flat Table

For tries (as used by PathMap), lookup cost is $O(d)$ where d is path depth, versus $O(1)$ for flat hash tables. The trade-off:

```

theorem trie_wins_when_cost_bounded
  (flatSize trieNodes depth queries : Nat)
  (h_depth : depth ≥ 1)
  (h_trie_smaller : trieNodes < flatSize)
  (h_cost_bounded : queries * (depth - 1) < flatSize - trieNodes) :
  trieQueryCost trieNodes depth queries <
  flatTableQueryCost flatSize queries

```


Guidance: Trie storage wins when paths share prefixes (reducing `trieNodes`) and depth is small. For PathMap-backed spaces with highly structured atoms (many shared prefixes), this is typically satisfied.

22.5 Formalized Files

The cost analysis is mechanized in:

- `Mettapedia/Complexity/FiniteFunctionStorage.lean` — core crossover theory
- `Mettapedia/Complexity/SymbolicComputation.lean` — arithmetic costs
- `Mettapedia/Complexity/PathMapCost.lean` — trie integration

All theorems are proven without axioms or `sorry`. The analysis informs CeTTa/MORK design: use PathMap storage for heavy query workloads and reverse queries; compute on demand for large domains or infrequent access.

Acknowledgements

The MORK/MM2 design is due to Meredith, Stay, and collaborators [1]. Luke Peterson designed and implemented the MORK kernel and MM2 language. Adam Vandervorst created the PathMap data structure and its Rust implementation. Remy Clarke contributed MM2 structuring patterns and example programs. The Lean 4 formalization was developed with AI assistance (Claude Code, ChatGPT, Codex).

References

- [1] L. G. Meredith, M. Stay, et al. *MORK: MeTTa Optimal Reduction Kernel*. Design notes and Rust implementation, 2020–2026.
- [2] Zar and Oruži. Process calculi formalized in Lean 4: Rho, Pi, MQ, and modal mu-calculus with cross-calculus bridges. Technical note, March 2026.
- [3] G. Huet. The Zipper. *Journal of Functional Programming*, 7(5):549–554, 1997.
- [4] C. McBride. The derivative of a regular type is its type of one-hole contexts. Unpublished manuscript, 2001.
- [5] M. Williams and M. Stay. Native Type Theory. In *Applied Category Theory (ACT)*, 2021.
- [6] L. G. Meredith and M. Stay. Operational Semantics in Logical Form. Technical report.
- [7] B. Goertzel, M. Iklé, I. F. Goertzel, and A. Heljakka. *Probabilistic Logic Networks*. Springer, 2009.
- [8] R. J. Solomonoff. A formal theory of inductive inference, Part I. *Information and Control*, 7(1):1–22, 1964.
- [9] T. J. Green, G. Karvounarakis, and V. Tannen. Provenance semirings. In *PODS*, pages 31–40, 2007.

- [10] I. S. Mumick, H. Pirahesh, and R. Ramakrishnan. The magic of duplicates and aggregates. In *VLDB*, pages 264–277, 1990.
- [11] S. Abiteboul, R. Hull, and V. Vianu. *Foundations of Databases*. Addison-Wesley, 1995.

Table 2: PathMap library files.

File	Key Content	Lines
<i>Core Algebraic Theory</i>		
Core.lean	AlgebraicResult, lattice hierarchy, Finset instance	440
CoinductiveTrie.lean	CTrie V, pointwise operations, bisimilarity laws	226
FiniteTrie.lean	FTrie V, operations, upsertChild, graftAtPath	470
TrieRefinement.lean	Sorted, join_lookup, toCTrie	243
TriePathMapInstance.lean	PathMapQuantale (FTrie V), JoinIdem, MeetIdem	129
<i>Zipper Hierarchy and ZAM</i>		
Zipper.lean	6-class typeclass hierarchy (ZipperMoving...ZipperComplexity)	482
ZipperExecution.lean	ZipperIterationSound, ZAM soundness	216
FlatZipperInstance.lean	Reference list-cursor zipper, all contracts proved	117
TrieZipper.lean	SimpleTrieZipper V, trie-backed soundness	148
ZamContracts.lean	ZAM soundness contracts, compositionality	193
HuetZipper.lean	Huet zipper, round-trip, focus_lookup_general	324
<i>Morphisms and Correctness</i>		
Morphisms.lean	cata/cataChildren, count/depth/hash algebras	149
Anamorphism.lean	ana/anaChildren, singleton coalgebra	81
MorphismCorrectness.lean	cata_recCount, cata_rebuild_lookup	167
GraftCorrectness.lean	setValueAt_lookup, ftrieWritingSpec_root	87
SortedPreservation.lean	singleton_lookup_ne, sortedness infrastructure	68
UnitBridge.lean	FTrie Unit $\leftrightarrow$ path sets, fromPathList	132
<i>OSLF Instance and Evidence Bridges</i>		
OSLFInstance.lean	pathMapLang, 10 rewrite rules, NTT completeness	853
RelationBridge.lean	RelationalSpace typeclass, toRelationEnv	200
PLNBridge.lean	finsetPathEvidence, K&S sum rule, disjoint additivity	275
SolomonoffBridge.lean	Weighted evidence, Solomonoff posterior additivity	239
Measure.lean	PathMapValuation, counting/weighted instances	143
WorldModelBridge.lean	BinaryWorldModel via Multiset lifting	151
<i>HE Backend Bridge</i>		
ZipperWritingLaws.lean	In-place ops, InPlaceStatusSpec	118
HEBridge.lean	supportToTrie, same_support_same_trie	126
HEInterface.lean	Monad morphism chain, SpaceQuerySupport	152
CandidateArchitecture.lean	Two-phase correctness, zero-crossing, invalidation	481
Total		7,165

Table 3: Conformance test suite. All theorems are proved by `rfl`.

#	Feature	MM2 Source	Theorem
1	Expression add/remove	<code>test_add_simple.mm2</code>	<code>conformance_test1_add_simple</code>
2	Flat symbol add	<code>test_add_constant.mm2</code>	<code>conformance_test2_add_constant</code>
3	Variable binding	<code>test3_var_binding.mm2</code>	<code>conformance_test3_var_binding</code>
4	Conjunctive (shared var)	<code>test4_conjunctive.mm2</code>	<code>conformance_test4_conjunctive</code>
5	Equality (<code>\$x \$x</code>)	<code>test5_equal_pair.mm2</code>	<code>conformance_test5_equal_pair</code>
6	Pattern mismatch	<code>test6_no_match.mm2</code>	<code>conformance_test6_no_match</code>
7	Nested expressions	<code>test7_nested.mm2</code>	<code>conformance_test7_nested</code>
8	Multi-step fixpoint	<code>test8_multi_step.mm2</code>	<code>conformance_test8_*</code> (5 thms)
9	Arity mismatch	(inline)	<code>conformance_test9_arity_mismatch</code>

Table 4: Formal coverage boundary.

Aspect	Formalized	Not Formalized
MM2 syntax	add/remove/head sinks, FoldAggregator, SourceFactor	Remaining 11 Rust sink types
Pattern matching	Soundness + completeness (expressions), computable evaluator	Coreferential matching
Space transitions	Yes + sink persistence	Nested collection rewrite
Three-phase protocol	Priority bands + aggregator + phase $\leftrightarrow$ scheduler exactness	—
Work-queue scheduler	Read-copy, self-respawn, ordering, full correspondence	Concurrent scheduling
Source-side input	BTM, == (equality), != (inequality)	ACT (external file)
Oracle semantics	Z3 query routing, payload-to-space, oracle pattern matching, mock conformance	Z3 subprocess lifecycle, SMT-LIB parsing
Conformance testing	Kernel-checked theorems, 9+ MM2 tests	—
PathMap algebra	Quantale hierarchy + laws (Finset, CTrie, FTrie instances)	—
PathMap trie	Join/meet/subtract/restrict + refinement	Full commutativity (non-Unit), concrete node types (Dense/Line/Tiny)
PathMap zipper	6-class typeclass hierarchy, ZAM soundness, 2 concrete instances	Mutation ops (ZipperWriting), product/overlay zippers, conflict detection
PathMap serialization	—	Paths serialization (19k Rust LOC), ACT on-disk format, round-trip correctness
PathMap morphisms	—	Catamorphism/anamorphism, fusion laws
PathMap OSLF instance	<code>pathMapLang</code> , 10 rewrite rules, NTT completeness	—
PathMap evidence	PLN BinaryEvidence, Solomonoff weighting, K&S valuations, world-model bridge	Full BinaryWorldModel on Finset
MORK-PathMap bridge	Algebraic (join/subtract)	—
MeTTaIL bridge	Flat rules, multi-premise, freshness guards, authored-context zipper/lens compilation, extended bridge (<code>.subst</code> /rest-var)	Native path-focused execution optimization
PLN routing witnesses	6 rules (transitivity, MP, abduction, guarded)	—
MQ bridge	Binary + N-ary fold $\leftrightarrow$ COMM	Real process witnesses
Compat-head compilation	Invertible fragment boundary, staged inverse, forward/reverse roundtrip (61 thms)	PeTTa Dispatch bridge, non-injective heads
Direct-exec surface	Scheduler exactness, PeTTa eval fragment	add-atom/remove-atom bridge, collection patterns, beta-reduction
Execution boundary	<code>morkTranslatable</code> packaging, pattern normalization, <code>morkTranslatable_applyBindings</code>	—
Runtime kernel packaging	Effect-class bridge, classified fragments, memoization safety, resource descriptors (4 resources)	Named atomspace exec, ACT exec seam
MM2 bridge round-trip	<code>mm2 raise lower</code> ,	Context-sensitive lowering

Table 5: Runtime resource descriptors.

Resource	Kernel Classes	Effect	Backend	Proved?
<code>defaultAtomSpace</code>	<code>ruleExec, query, spaceEffect</code>	<code>writesState</code>	MORK/MM2	yes
<code>namedAtomSpace</code>	(none)	<code>writesState</code>	none	no
<code>solverResource</code>	<code>oracle</code>	<code>oracleIO</code>	MORK/MM2	no
<code>externalResource</code>	<code>oracle</code>	<code>oracleIO</code>	MORK/MM2	no